



RAPPORT D'ÉTUDE

Rapport De Projet Tutoré Interactif Git (Fork & Pull request)

**Formation : But Sciences de Données -----1 ère
Année**

Année universitaire 2025-2026

Réalisé par :

M'mahawa doss camara

TABLE DES MATIÈRES

1. Introduction	2
2. Concepts fondamentaux.....	3
2.1 Qu'est-ce qu'un dépôt (Repository) ?.....	3
2.2 Le Fork	3
2.3 Les branches	3
2.4 La Pull Request (PR).....	3
3. Workflow complet : Fork → Pull Request.....	4
3.1 Étape 1 — Forker le dépôt original	4
3.2 Étape 2 — Cloner le fork en local	4
3.3 Étape 3 — Créer une branche de travail.....	4
3.4 Étape 4 — Effectuer les modifications	4
3.5 Étape 5 — Committer les modifications	4
3.6 Étape 6 — Pousser vers le fork	4
3.7 Étape 7 — Ouvrir une Pull Request.....	4
4. Projet Python — GitQuest Visualiseur d'arbres Git	5
4.1 Présentation générale.....	5
4.2 Architecture du projet.....	5
4.3 Architecture des modules	5
4.4 Modes d'utilisation	6
4.6 Points techniques notables	6
4.6.1 Algorithme de tri topologique (Kahn).....	6
4.6.2 Comparaison topologique indépendante des hashes.....	7
4.6.3 Compatibilité Windows.....	7
5. Fonctionnalité	7
5.1 Besoin identifié	7
5.2 Solution implémentée	7
5.2.1 Workflow enseignant — créer une question.....	7
5.2.2 Workflow étudiant — résoudre une question.....	8
5.3 Format du fichier de question (JSON).....	8
5.4 Diagramme de séquence.....	8
5.5 Robustesse et gestion des erreurs	9
6. Récapitulatif des commandes Git	9
7. Résultats et apprentissages	10
7.1 Résultats obtenus.....	10
7.1.1 Captures d'écran de l'outil.....	10
7.2 Difficultés rencontrées	11
7.3 Compétences développées	11
8. Conclusion	13
9. Références.....	14

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du projet tutoré de BUT Science des Données (1^{ère} année). Il recouvre deux volets complémentaires : d'une part, la maîtrise pratique du workflow collaboratif Git/GitHub à travers le cycle Fork → Clone → Branch → Push → Pull Request ; d'autre part, le développement d'un outil Python original — GitQuest — permettant de visualiser les arbres Git dans le terminal et en fenêtre graphique.

Git est un système de contrôle de version distribué créé par Linus Torvalds en 2005. GitHub est la plateforme d'hébergement de code qui s'est imposée comme standard dans l'industrie du développement logiciel. La maîtrise de ces outils est aujourd'hui indispensable dans le domaine de la Science des Données, où la collaboration sur des notebooks Jupyter, des scripts Python ou des pipelines de données nécessite un versioning rigoureux.

GitQuest est un projet Python développé spécifiquement pour ce cours. Il permet à la fois de visualiser l'arbre de commits d'un dépôt réel, et de proposer un mode pédagogique interactif dans lequel l'étudiant doit reproduire un état Git cible. Ce rapport documente l'ensemble du workflow mis en œuvre, l'architecture du projet Python, ainsi que la nouvelle fonctionnalité destinée aux enseignants pour créer des questions personnalisées.

2. Concepts fondamentaux

2.1 Qu'est-ce qu'un dépôt (Repository) ?

Un dépôt (ou repository) est un espace de stockage qui contient l'ensemble des fichiers d'un projet ainsi que tout l'historique des modifications effectuées sur ces fichiers. Il en existe deux types :

- Dépôt local : situé sur la machine du développeur, accessible sans connexion réseau ;
- Dépôt distant (remote) : hébergé sur une plateforme comme GitHub, accessible depuis n'importe quelle machine connectée.

2.2 Le Fork

Le fork est l'opération qui consiste à créer une copie indépendante d'un dépôt existant sur son propre compte GitHub. Contrairement au simple clonage, le fork maintient un lien avec le dépôt d'origine, ce qui permet d'échanger des modifications via les Pull Requests. Il faut distinguer trois notions :

Opération	Localisation	Description
Fork	GitHub (distant)	Copie du dépôt sur votre compte GitHub — lien maintenu avec l'original
Clone	Local (votre machine)	Téléchargement du dépôt (fork ou original) sur votre ordinateur
Remote origin	Référence Git locale	Alias pointant vers l'URL du fork ; évite de retaper l'URL complète

2.3 Les branches

Une branche est une ligne de développement indépendante qui permet de travailler sur une fonctionnalité ou une correction sans impacter la branche principale (main). La commande suivante crée une branche et bascule dessus en une seule opération :

```
$ git checkout -b nom-de-la-branche
```

2.4 La Pull Request (PR)

Une Pull Request est une demande formelle adressée au propriétaire d'un dépôt pour qu'il intègre les modifications réalisées dans un fork ou une branche. C'est le mécanisme central de la contribution dans les projets open source et collaboratifs. Elle permet la revue de code, les commentaires et la traçabilité des changements.

3. Workflow complet : Fork → Pull Request

3.1 Étape 1 — Forker le dépôt original

Sur GitHub, naviguer vers le dépôt cible et cliquer sur le bouton Fork situé en haut à droite. GitHub affiche une page de confirmation ; cliquer sur Create fork. Le dépôt est alors copié sur le compte personnel.

Dépôt original : github.com/denis-migdal-Student-projects/GitQuest

Fork créé : github.com/hawa-cyber/studentprojetGit

3.2 Étape 2 — Cloner le fork en local

Une fois le fork créé sur GitHub, on le télécharge sur sa machine via la commande clone. Le lien HTTPS s'obtient en cliquant sur le bouton vert Code du fork :

```
$ git clone https://github.com/hawa-cyber/studentprojetGit.git
$ cd studentprojetGit
```

Lors du clonage, Git crée automatiquement un remote nommé origin qui pointe vers l'URL du fork.

3.3 Étape 3 — Créer une branche de travail

Il est fortement recommandé de ne jamais travailler directement sur la branche main. On crée donc une branche dédiée à la modification souhaitée :

```
$ git checkout -b feature/update-readme
```

3.4 Étape 4 — Effectuer les modifications

Les modifications ont été apportées au fichier README.md du dépôt GitQuest. Le contenu ajouté structure le projet de manière claire et professionnelle, avec les sections Description, Installation, Utilisation et Auteur.

3.5 Étape 5 — Committer les modifications

Après les modifications, on enregistre l'état du travail via un commit en suivant la convention Conventional Commits :

```
$ git add README.md
$ git commit -m "docs : mise à jour du README avec section installation"
```

3.6 Étape 6 — Pousser vers le fork

```
$ git push origin feature/update-readme
```

3.7 Étape 7 — Ouvrir une Pull Request

Sur GitHub, dans la page du fork, un bouton Compare & pull request apparaît automatiquement. En cliquant dessus, on accède au formulaire de création de PR avec un titre

et une description détaillée. Le PR est alors visible dans l'onglet Pull requests du dépôt original et peut être consultée, commentée ou acceptée (fusion) par le propriétaire du dépôt.

4. Projet Python — GitQuest Visualiseur d'arbres Git

4.1 Présentation générale

GitQuest est un outil Python développé dans le cadre de ce projet tutoré. Il permet de visualiser l'arbre des commits d'un dépôt Git sous deux formes : un affichage encadré dans le terminal avec trois colonnes alignées (références, arbre graphique, hash + message), et une fenêtre graphique interactive avec matplotlib où chaque branche est colorée distinctement.

L'outil propose également un mode pédagogique interactif : l'étudiant voit côte à côte l'état actuel de son dépôt et l'état cible à atteindre. Il entre des commandes Git une par une, et l'outil vérifie automatiquement si l'objectif est atteint.

4.2 Architecture du projet

Le projet est structuré en six modules Python avec des responsabilités clairement séparées :

Module	Rôle
<code>main.py</code>	Point d'entrée — gère les 5 modes d'utilisation (démonstration, affichage, interactif, graphique, enseignant)
<code>git_tree.py</code>	Structure de données GitTree et algorithme de tri topologique de Kahn pour ordonner les commits
<code>git_reader.py</code>	Lecture d'un vrai dépôt Git via subprocess — compatible Windows et Linux
<code>display.py</code>	Affichage encadré dans le terminal avec la bibliothèque rich (3 colonnes, couleurs)
<code>git_graph.py</code>	Affichage graphique avec matplotlib — fond sombre, une couleur par branche
<code>comparaison.py</code>	État cible par défaut et comparaison topologique (indépendante des hashes)
<code>question.py</code>	NOUVEAU — Outil enseignant : sauvegarde et chargement d'états cibles en JSON

4.3 Architecture des modules

Le schéma ci-dessous représente les dépendances entre les sept modules du projet. `main.py` est le point d'entrée central qui orchestre tous les autres. Les imports à chargement différé (« lazy ») — `git_reader`, `git_graph`, `question` — ne sont chargés que lorsque le mode

correspondant est activé, ce qui évite de pénaliser les modes rapides (démonstration, affichage simple) par des chargements inutiles.

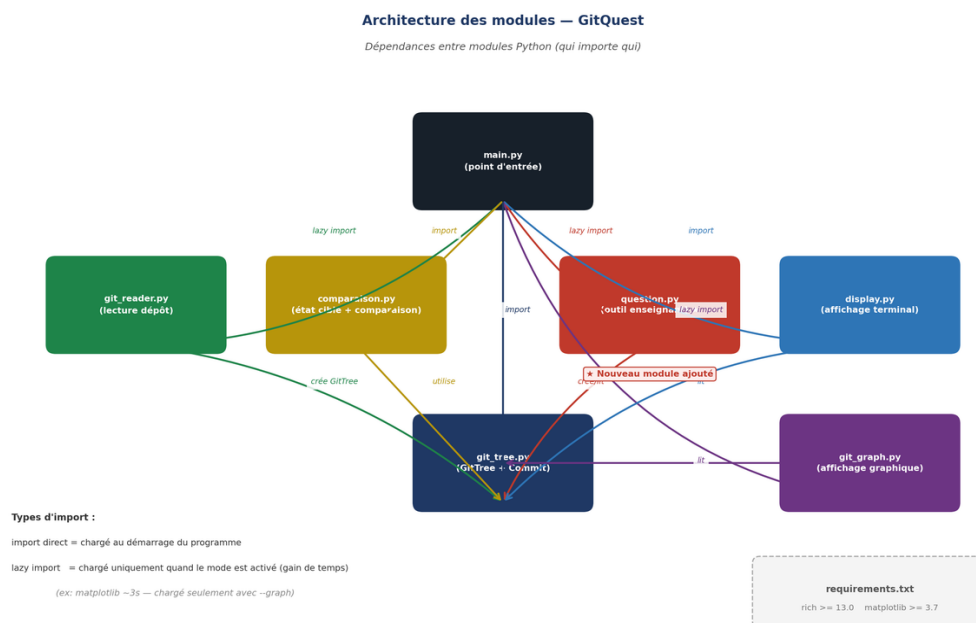


Figure 2 : Architecture des modules — dépendances et imports

4.4 Modes d'utilisation

Commande	Description
<code>python main.py</code>	Démonstration avec données fictives (ne nécessite pas de dépôt Git)
<code>python main.py <depot></code>	Affiche l'arbre d'un dépôt Git existant en 3 colonnes
<code>python main.py <depot> --loop</code>	Mode interactif avec l'état cible par défaut
<code>python main.py <depot> --graph</code>	Visualisation graphique matplotlib
<code>python main.py <depot> --save-target f.json</code>	[Enseignant] Sauvegarde l'état comme question
<code>python main.py <depot> --loop --target f.json</code>	[Étudiant] Résout une question personnalisée

4.6 Points techniques notables

4.6.1 Algorithme de tri topologique (Kahn)

Pour afficher les commits dans le bon ordre chronologique (du plus récent au plus ancien) en tenant compte des branches et des merges, GitTree utilise l'algorithme de Kahn. Cet algorithme de tri topologique garantit qu'un commit parent est toujours affiché avant ses enfants, quel que soit l'ordre dans lequel Git renvoie les commits.

4.6.2 Comparaison topologique indépendante des hashes

La comparaison entre l'état actuel d'un dépôt et l'état cible ne peut pas se baser sur les hashes des commits, car deux dépôts différents ayant le même historique (mêmes messages, même structure de branches) auront nécessairement des hashes différents. GitQuest calcule donc une forme canonique de chaque commit, qui encode récursivement son message et la structure de tous ses parents. Deux arbres sont considérés identiques si leurs branches portent les mêmes noms et si leurs formes canoniques correspondent.

4.6.3 Compatibilité Windows

Le module `git_reader.py` contient une détection automatique de l'exécutable Git sur Windows, en cherchant d'abord dans le PATH système puis dans les emplacements d'installation classiques (`Program Files\Git`). L'encodage UTF-8 est forcé sur la sortie standard pour éviter les erreurs cp1252 dans PowerShell.

5. Fonctionnalité

5.1 Besoin identifié

Une fonctionnalité manquante a été identifiée : la possibilité de créer ses propres états cibles (questions) sans modifier le code source. Dans la version initiale, l'état cible était codé en dur dans `comparaison.py`, ce qui imposait de modifier le fichier Python pour changer la question une approche peu pratique dans un contexte pédagogique.

Le besoin exprimé était le suivant :

- Il existe un état actuel du dépôt de l'étudiant ;
- Il existe un état cible qui, une fois atteint, signifie que la question est résolue ;
- Comment puis-je créer un état cible à distribuer aux étudiants ?

5.2 Solution implémentée

Un nouveau module `question.py` a été développé pour répondre à ce besoin. Il s'appuie sur un format de fichier JSON simple et lisible, que l'enseignant peut créer automatiquement ou manuellement.

5.2.1 Workflow enseignant — créer une question

L'enseignant suit ces trois étapes simples :

- Étape 1 : Préparer un dépôt Git dans l'état que les étudiants devront reproduire (créer des branches, effectuer des commits, créer des tags, etc.) ;
- Étape 2 : Lancer la commande de sauvegarde pour exporter cet état en JSON ;
- Étape 3 : Distribuer le fichier JSON aux étudiants.

Commande de sauvegarde :

```
python main.py <chemin_depot> --save-target question1.json
```

Avec une description de l'objectif :

```
python main.py <chemin_depot> --save-target q1.json --desc "Créer 2 branches avec 3 commits"
```

5.2.2 Workflow étudiant — résoudre une question

L'étudiant reçoit le fichier JSON et lance le mode interactif avec la question de son enseignant :

```
Python main.py <chemin_depot_etudiant> --loop --target question1.json
```

L'outil affiche côte à côte l'état actuel du dépôt de l'étudiant et l'état cible défini par l'enseignant. L'étudiant entre des commandes Git une par une. Lorsque l'état cible est atteint, la session se termine automatiquement avec le message : « Objectif atteint ! ».

5.3 Format du fichier de question (JSON)

Le fichier JSON est volontairement simple et lisible. Il peut aussi bien être généré automatiquement que créé manuellement par l'enseignant :

```
{
  "description » : "Créer une branche feature avec 2 commits",
  "commits": [
    {
      "hash":      "a1b2c3d",
      "message": "Initial commit",
      "parents » : [],
      "refs : ["tag : v1.0"]
    },
    {
      "hash :      "b2c3d4e",
      "message : "Ajout README",
      "parents" : ["a1b2c3d"],
      "refs" : ["HEAD -> main"]
    }
  ]
}
```

5.4 Diagramme de séquence

Le diagramme ci-dessous illustre les interactions entre les différents composants du système lors des deux phases de la fonctionnalité enseignant : la création d'une question (--save-target) et sa résolution par l'étudiant (--loop --target).

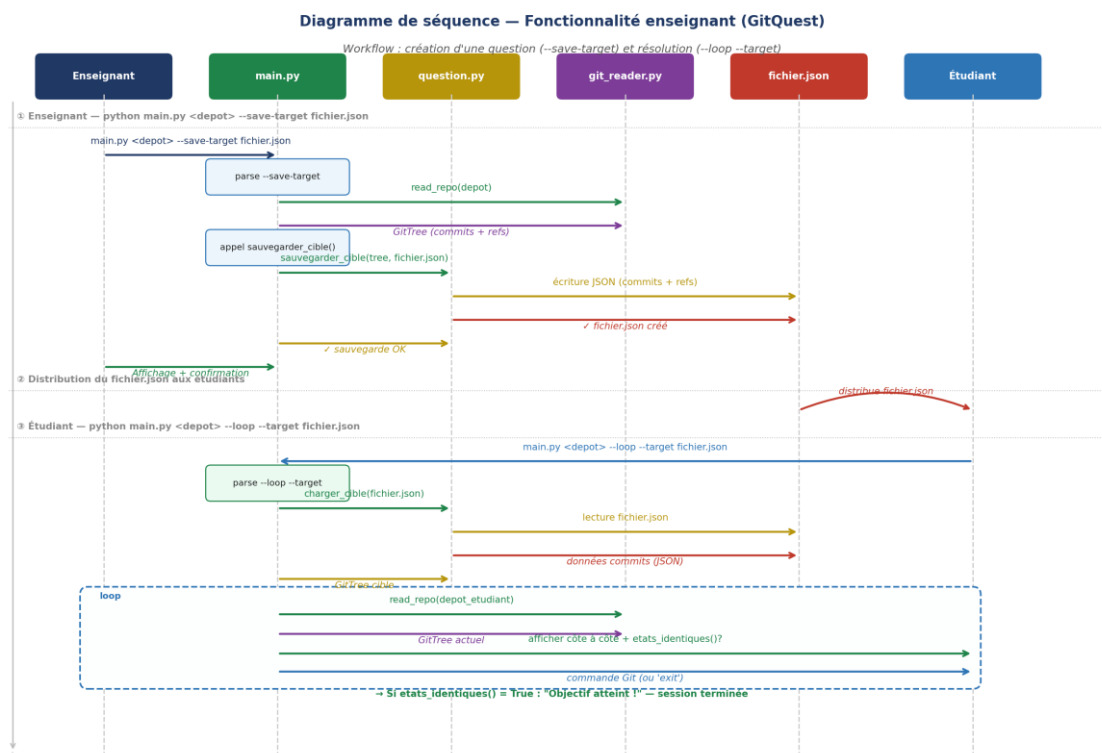


Figure 1 : Diagramme de séquence — Création et résolution d'une question (GitQuest)

5.5 Robustesse et gestion des erreurs

Le module question.py intègre une gestion complète des erreurs : fichier introuvable, JSON malformé, champs manquants dans un commit. Chaque erreur produit un message clair indiquant à l'enseignant ou à l'étudiant exactement ce qui ne va pas et comment le corriger.

Cas d'erreur	Message affiché
Fichier JSON introuvable	Fichier de question introuvable : <chemin> — Vérifiez le chemin ou créez-en un avec --save-target
JSON malformé	Fichier JSON invalide (<chemin>) : <détail de l'erreur>
Clé 'commits' absente	Format de fichier invalide : clé 'commits' manquante ou incorrecte
Hash ou message manquant	Commit invalide dans <chemin> : clé 'hash' (ou 'message') manquante
--target sans nom de fichier	Usage affiché avec exemple correct

6. Récapitulatif des commandes Git

Commande	Description
<code>git clone <url></code>	Télécharge un dépôt distant sur la machine locale

Commande	Description
<code>git checkout -b <nom></code>	Crée une nouvelle branche et bascule dessus
<code>git add <fichier></code>	Ajoute un fichier à l'index (staging area)
<code>git commit -m "msg"</code>	Enregistre les modifications dans l'historique
<code>git push origin <branche></code>	Envoie les commits vers le dépôt distant (fork)
<code>git pull</code>	Récupère et intègre les modifications distantes
<code>git log --oneline --graph</code>	Affiche l'historique sous forme d'arbre condensé
<code>git branch</code>	Liste toutes les branches locales
<code>git status</code>	Affiche l'état du répertoire de travail
<code>git diff</code>	Montre les différences non encore committées

7. Résultats et apprentissages

7.1 Résultats obtenus

Au terme de ce projet, l'ensemble des objectifs fixés a été atteint :

- Maîtrise complète du workflow Fork / Clone / Branch / Push / Pull Request ;
- Installation et configuration de Git sur environnement Windows (PowerShell) ;
- Développement d'un outil Python fonctionnel (GitQuest) avec 7 modules ;
- Implémentation d'un algorithme de tri topologique (Kahn) pour l'affichage des commits ;
- Développement d'un comparateur topologique indépendant des hashes ;
- Ajout de la fonctionnalité enseignant (`--save-target / --target`) ;
- Soumission d'une demande d'intégration (Pull Request) sur le dépôt GitQuest d'origine.

7.1.1 Captures d'écran de l'outil

Les figures suivantes illustrent le rendu réel de GitQuest dans les deux modes d'affichage principaux.

```

$ python main.py

┌─ demo -- donnees fictives ─┐
| HEAD → main • b2c3d4e Correction bug |
| | • a1b2c3d Nouvelle fonction |
| | | / |
| □ feature • c3d4e5f Ajout README |
| ♦ v1.0 |
| | • d4e5f6g Initial commit |
└──────────────────────────┘

Affichage encadré en 3 colonnes : références | arbre | hash + message

```

Figure 4 : Mode démo — Affichage encadré en 3 colonnes (terminal, fond sombre)

```

$ python main.py . --loop --target question.json

État actuel                                État cible (objectif)
┌────────────────────────────────┐      ┌────────────────────────────────┐
| HEAD → main • abc1234 Initial commit |  | HEAD → main • b2c3d4e Correction bug | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | |  | | • a1b2c3d Nouvelle fonction |
| | | | | | | | | | | | | | | | | | |  | | | / |
| | | | | | | | | | | | | | | | | | |  | | ♦ v1.0 • c3d4e5f Initial commit |
└────────────────────────────────┘      └────────────────────────────────┘
                                     ⇄

>>> Commande (ou 'exit') : git checkout -b feature

```

Figure 5 : Mode interactif — État actuel et état cible affichés côte à côte

7.2 Difficultés rencontrées

Plusieurs difficultés ont été rencontrées et surmontées au cours de ce projet :

- Installation de Git sous Windows : la commande git n'était pas reconnue dans PowerShell immédiatement après l'installation, nécessitant un redémarrage du terminal. Solution : détection automatique implémentée dans git_reader.py ;
- Navigation dans les répertoires : l'absence de .git dans le répertoire courant indiquait que le terminal n'était pas positionné dans le bon dossier ;
- Confusion entre git branch -b (n'existe pas) et git checkout -b (commande correcte) ;
- Comparaison d'arbres Git : les hashes diffèrent naturellement entre dépôts, même pour un historique identique. Solution : comparaison par formes canoniques récursives.

7.3 Compétences développées

Ce projet a permis de développer des compétences techniques et transversales essentielles dans le domaine de la Science des Données et du développement logiciel collaboratif :

-
- Gestion de version avec Git et collaboration via GitHub ;
 - Développement Python orienté objet avec séparation des responsabilités (modules) ;
 - Algorithmique : tri topologique, comparaison structurelle d'arbres ;
 - Sérialisation JSON pour la persistance des données ;
 - Communication technique : documentation de code (docstrings), README, rapport.

8. Conclusion

Ce projet tutoré a dépassé le cadre initial d'une simple initiation au workflow Git/GitHub. En développant GitQuest, nous avons mis en pratique des concepts avancés de la programmation Python — algorithmes de graphes, sérialisation JSON, gestion des processus système — tout en construisant un outil réellement utile dans un contexte pédagogique.

La fonctionnalité enseignant, ajoutée en réponse à un besoin identifié, illustre bien la dynamique de contribution open source que ce projet cherchait à enseigner : identifier un besoin, proposer une solution, l'implémenter proprement et la documenter. C'est exactement le cycle qu'une Pull Request incarne.

Ces apprentissages sont directement transposables dans les missions professionnelles à venir. En alternance ou en projet, Git est omniprésent. La maîtrise du workflow collaboratif, la capacité à lire et visualiser un historique de commits, et la compréhension des mécanismes de branches et de merge constituent des atouts concrets et immédiatement valorisables.

En perspective, GitQuest pourrait être enrichi par l'intégration de conflits de merge dans le mode interactif, par une interface web légère (Flask/Streamlit), ou par des GitHub Actions pour l'intégration continue des tests.

9. Références

- Dépôt original du projet GitQuest : <https://github.com/denis-migdal-Student-projects/GitQuest>
- Fork personnel : <https://github.com/hawa-cyber/studentprojetGit>
- Documentation officielle Git : <https://git-scm.com/doc>
- Documentation GitHub : <https://docs.github.com>
- Pro Git Book (Scott Chacon, Ben Straub): <https://git-scm.com/book/fr/v2>
- Documentation rich (affichage terminal) : <https://rich.readthedocs.io>
- Documentation matplotlib : <https://matplotlib.org/stable/contents.html>
- Conventional Commits : <https://www.conventionalcommits.org/fr/v1.0.0>